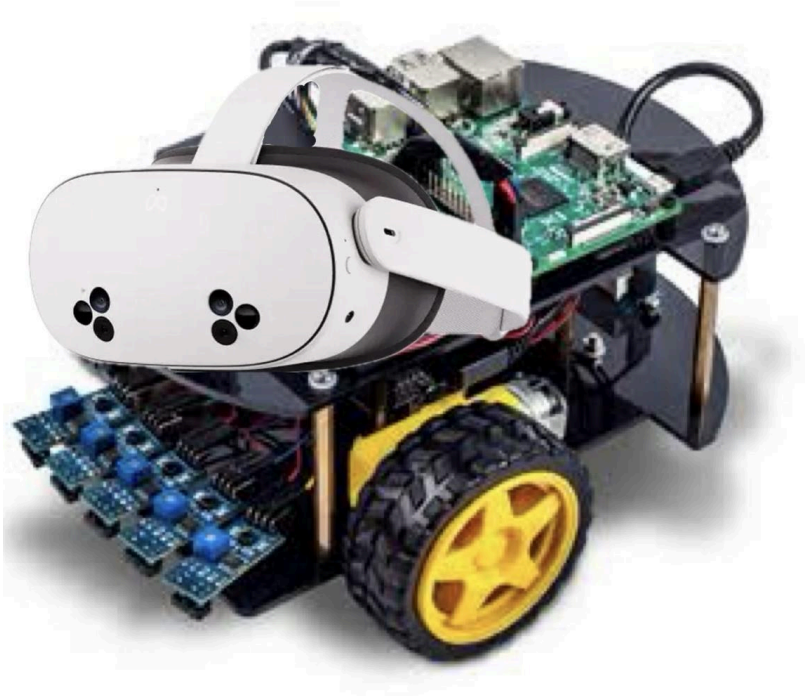
Virtual Robotics



Wesley Burcham, Matthew Palmer, Michael Brown
Client(s): Dr. Hall and Dr. Benedict
Computer Science Capstone 2026

Table of Contents

- **Introduction (pg. 2)**
- **Executive Summary (pg. 2)**
- **Scenarios (pg. 2 - 3)**
- **Interaction Model (pg. 4)**
- **Screen by Screen Breakdown (pg. 5-9)**
- **Errors and Edge Cases (pg. 10- 11)**
- **Technical Components (pg. 12-18)**
- **Glossary (pg. 18-24)**
- **Special Acknowledgments (pg. 24)**

Introduction:

- **Summary of the problem our project solves:**
  In robotics, there are many approaches to controlling a robot. Our clients, specifically college professors involved in robotics and mathematics, are looking for a control medium that allows them to maneuver the robot and observe sensor readings.

- **What our project is and how it solves the aforementioned problem:**
  The solution our team is implementing is a VR headset controller that can be used to control robot movement and observe sensor output. The main robot we'll be focusing on for this project is an OSOYOO Robot Pie Car equipped with a camera, a voltage meter, a tracking sensor, and an ultrasonic sensor. The headset we'll be using is the Meta Quest 3 equipped with 2 handheld joystick controllers and the actual headset. Our implementation will allow the user to control the robot via the joysticks and read from the camera and sensors using the headset.

Executive Summary:

The Virtual Robotics project is a WebXR Meta Quest application that will mainly aid Professor Hall's and Professor Benedict's research on real-world robotics applications. The main application functions are a live-feed robot camera and controller, as well as a menu to switch the inputs and outputs between multiple different robots to the headset (if a second robot is completed in time, which is outside the scope of this class' work).

The universality of the Pi robots will be a streamlined process that will only require small file modifications to be able to use a new robot that runs on a Raspberry Pi. After the completion of the project, all code and supplies used will be made public, allowing anybody to use, replicate, or expand upon our application.

The live camera footage of the robot camera, as well as the controls, will be used to efficiently control robots. It will consist of a start screen where you select any number of robots you will want to use from a menu selection screen highlighting them, and then a start button. Clicking the start button will then load the visuals of the first robot chosen, where you are shown the camera feed of the robot and a diagnostics see-through overlay that is on the side. This will have important information about the robot from the available sensors. You will also see a drop-down menu in which you will see an end session button to terminate the program along with settings to toggle the overlay on and off.

Use Cases and Scenarios:

In each of the following scenarios our actor can be a variety of different avatars. Our default is a researcher looking to expand the capabilities/use of VR in the context of robotics controls, but our user could just as easily be a recreationalist, a researcher of some other field

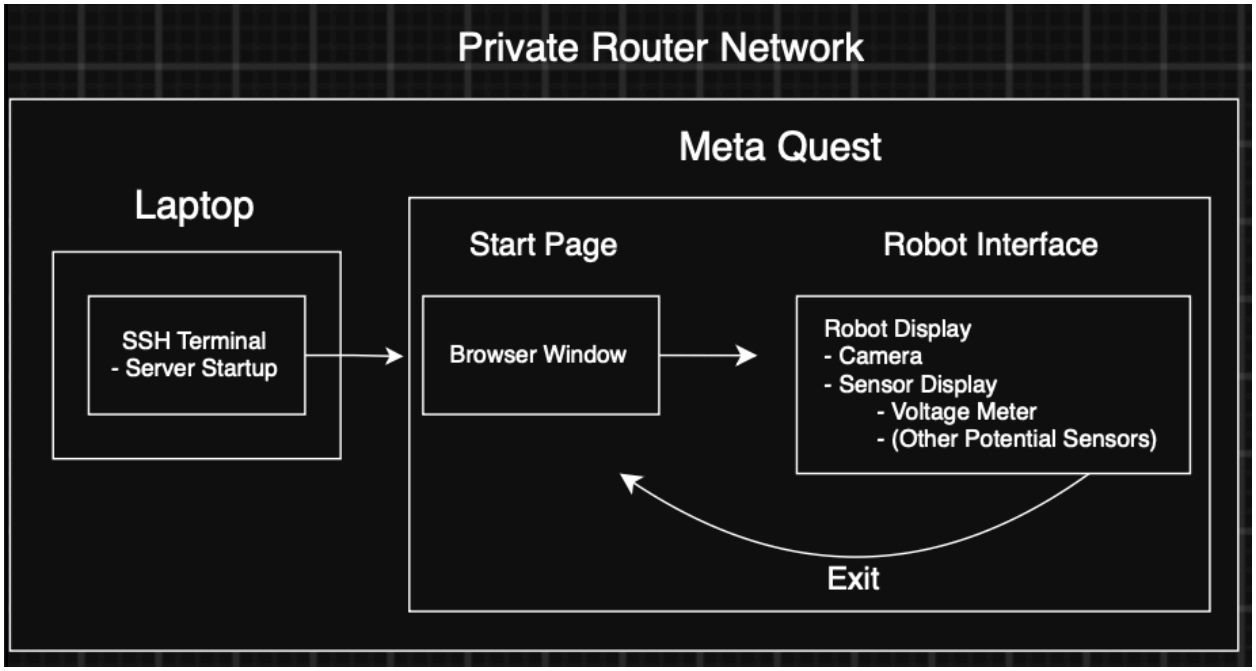
(environmentalist, wildlife biologist, etc), each with the goal of using the platform for its mobility and observational capabilities.

- **Platform Testing for Different Kinds of Robots:**
  Users can adapt the platform to their robot of choice and have a means of testing different systems of that robot with a platform that allows them to display any sensor readings. For example, like in our case, in order to test the circuitry and the ability of the motors to move in coordination with one another, we wrote a script that allowed the Meta Quest's joysticks to communicate with the robot and tell it to move forward. The success of this test allowed us to validate that there was nothing wrong with the hardware and that the robot was able to receive signals, operate off of them, and respond back over a digital network.

- **Environment Exploration**
  - Users of the platform can access a camera feed of whatever sensor on their robot of choice they'd like, and depending on how small their robot of choice is, they can even use the platform to gain access to certain areas or crevices that they'd have a hard time getting into by themselves.
  - If the user adds different kinds of sensors to the robot and connects them to the display, they can also use the platform to see things that humans can't. Examples include thermal/infrared sensors, night vision cameras, and various sensors that are used to see particles in the air that are normally invisible/undetectable to human senses.

- **Recreation**
  The platform can also be used for entertainment/noneducational purposes, such as having a sensor's camera feed be used for video footage that can be uploaded onto the internet as either part of a funny YouTube video or as makeshift security footage that can be used to spy on insects or small animals (e.g. fruit flies, ants, small dogs/cats, etc.).

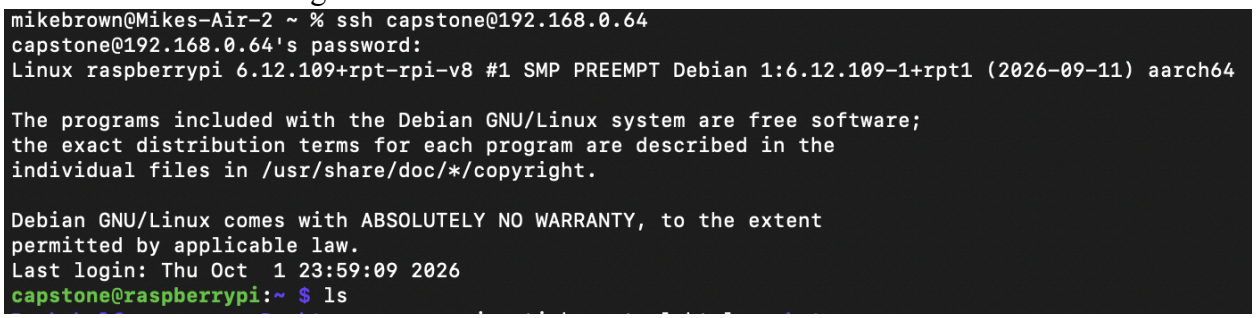
Interaction Model



Despite its simplicity, this diagram describes our entire system of interactions between our interfaces. In our system specifically, we used a separate router to minimize network congestion on our system's communications. This is not necessary, but we found it led to a very smooth experience from a networking perspective. Regardless, the session starts on a laptop we use to SSH into the Raspberry Pi so that we startup the servers that host our applications. Once started, the servers allow us to go into the Meta Quest and access the application via the browser through a URL. After that, all the user has to do is click a button that says "Enter Mixed Reality" and they're in. If they can see the display, and see that the car is responding to the joysticks, then everything is functioning normally.

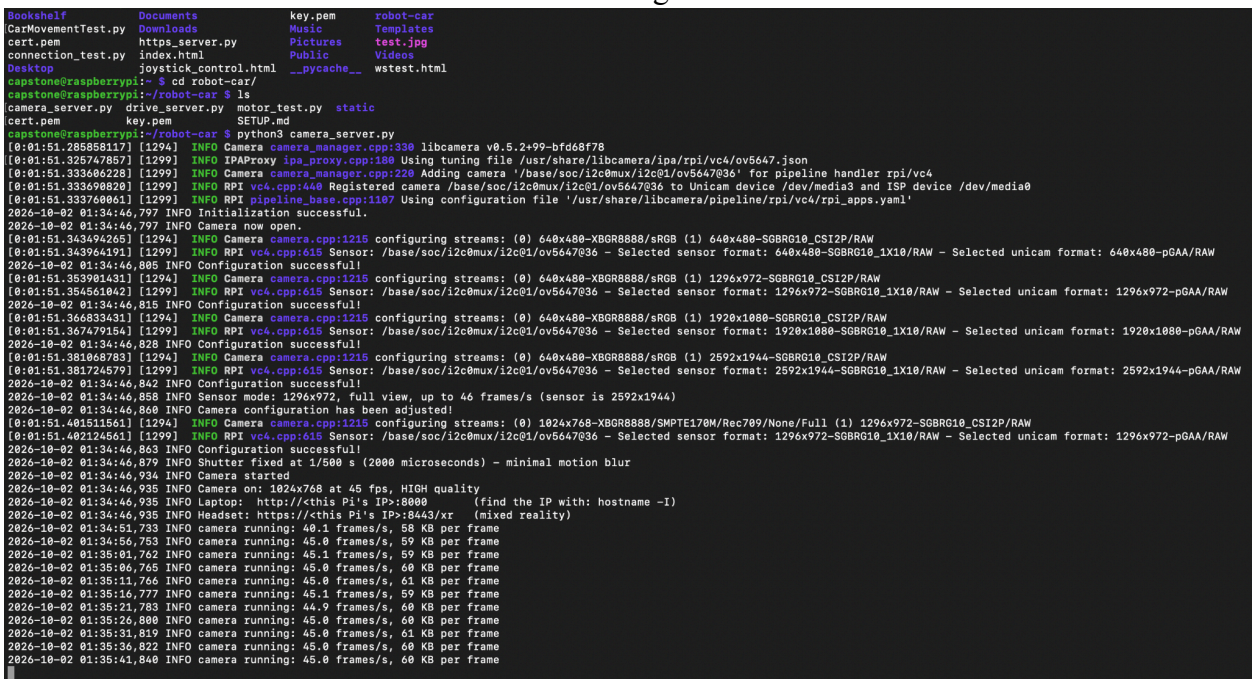
Screen by Screen Breakdown:

Terminal Screen showing SSH:



This is the first screen that's relevant to our application. From here (the terminal), the user is expected to ssh into the Raspberry Pi associated with the Robot Car using the command "ssh username@<Raspberry Pi's IP Address>" and entering the password. In our specific case, the command is "ssh capstone@192.168.0.64" and the password is MMWRobotCar. From here, you'll be entered into a terminal for the Raspberry Pi's system from which you can run the server.

SSH terminal screen with the camera server running:



This is an example of the camera server running with the output the user should see after running the command "python3/robot-car/camera_server.py". The output below is just text that assures the user that the camera and the pipeline responsible for sending it is functional, as well as the quality per frame of the most recent chunk of image streams. Another function of this server is to

load the certificates for the websocket so that they can be accessed from the Meta Quest browser (the browser requires the socket to be secured). This is also why this server should be started first. From there the user should open a second terminal, SSH into the Raspberry Pi's system, and do the following.

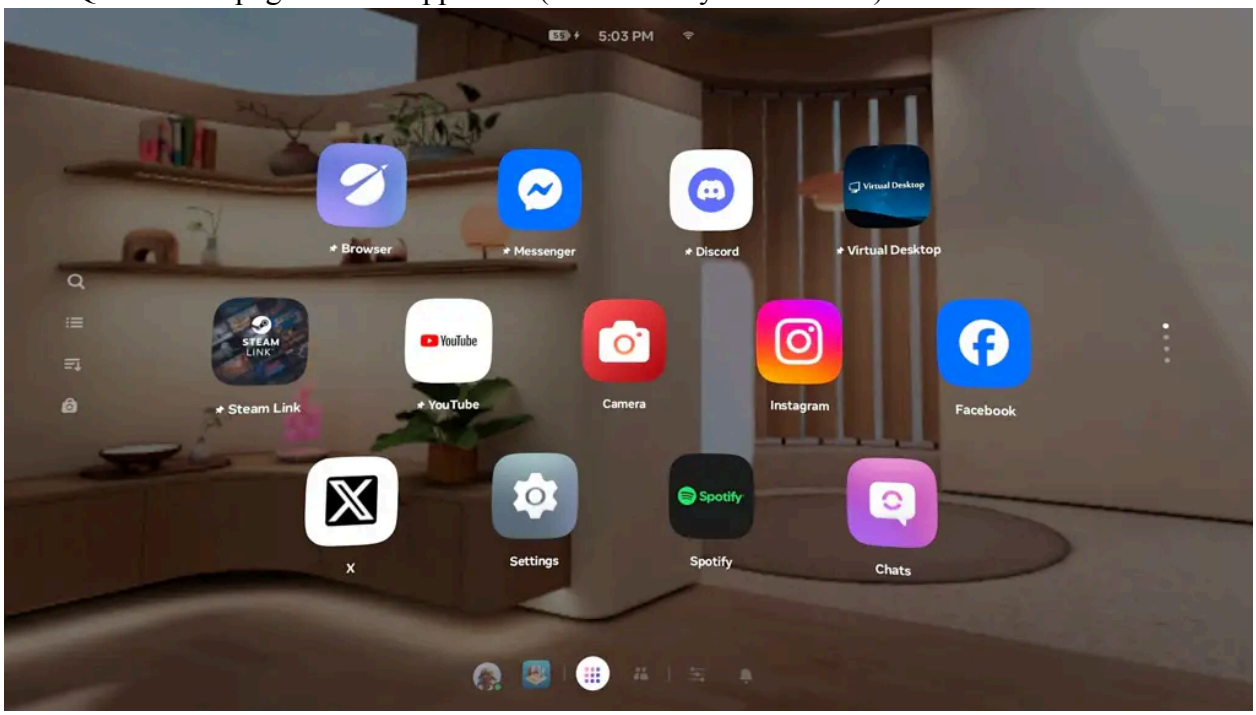
SSH Terminal with driving server running:

```
capstone@raspberrypi:~ $ cd robot-car/
capstone@raspberrypi:~/robot-car $ ls
camera_server.py  drive_server.py  motor_test.py  static
cert.pem          key.pem          SETUP.md
capstone@raspberrypi:~/robot-car $ python3 drive_server.py
LIVE — motors armed. Car on a block for the first run!
drive server listening on wss://0.0.0.0:8765
```

NOTE: THIS SHOULD BE A SECOND TERMINAL, NOT THE SAME ONE IN WHICH CAMERA_SERVER.PY IS RUNNING

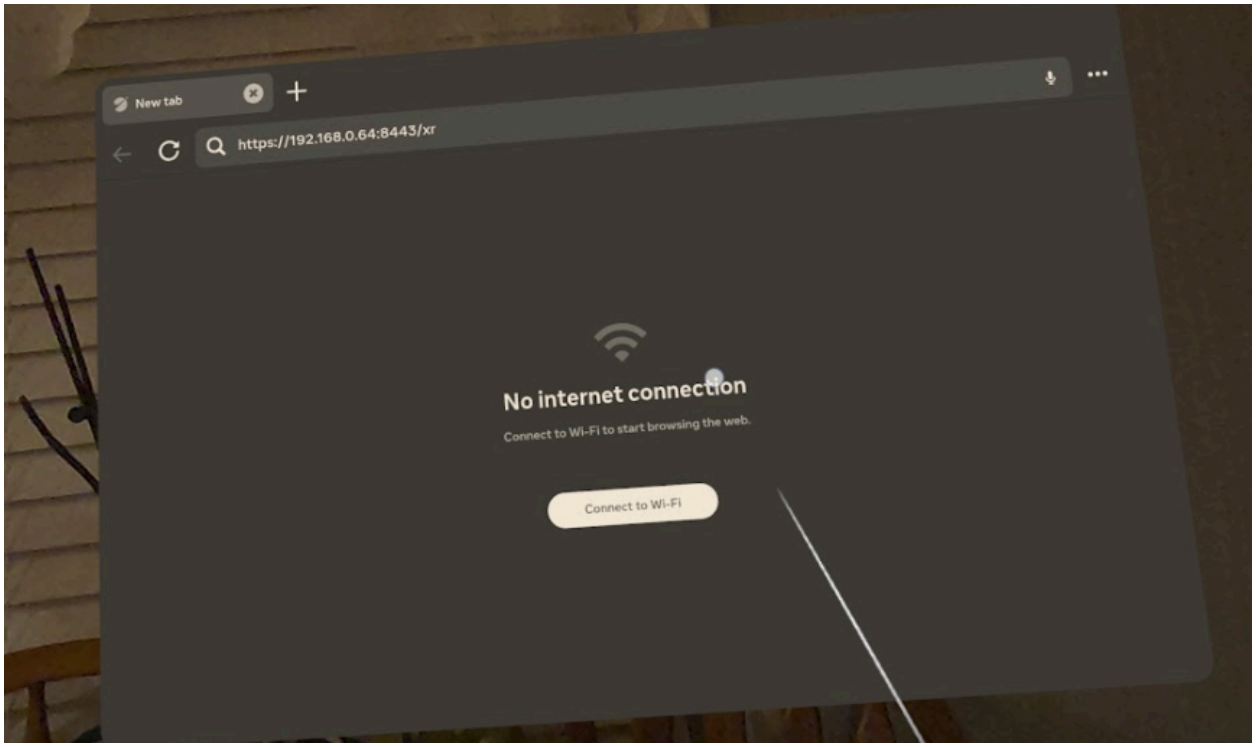
In this terminal the user is expected to run the server that hosts the driving commands, allowing the joystick input to be sent to and read by the Raspberry Pi and turn the wheels. From there, the user is expected to turn on the Meta Quest, and press the Meta Icon button on the right handed controller, which will bring up the following display. The text lets the user know that the controls are live, meaning it will move the car, and it also lets them know what port the service is listening on.

Meta Quest home page with all app icons (most notably the browser):



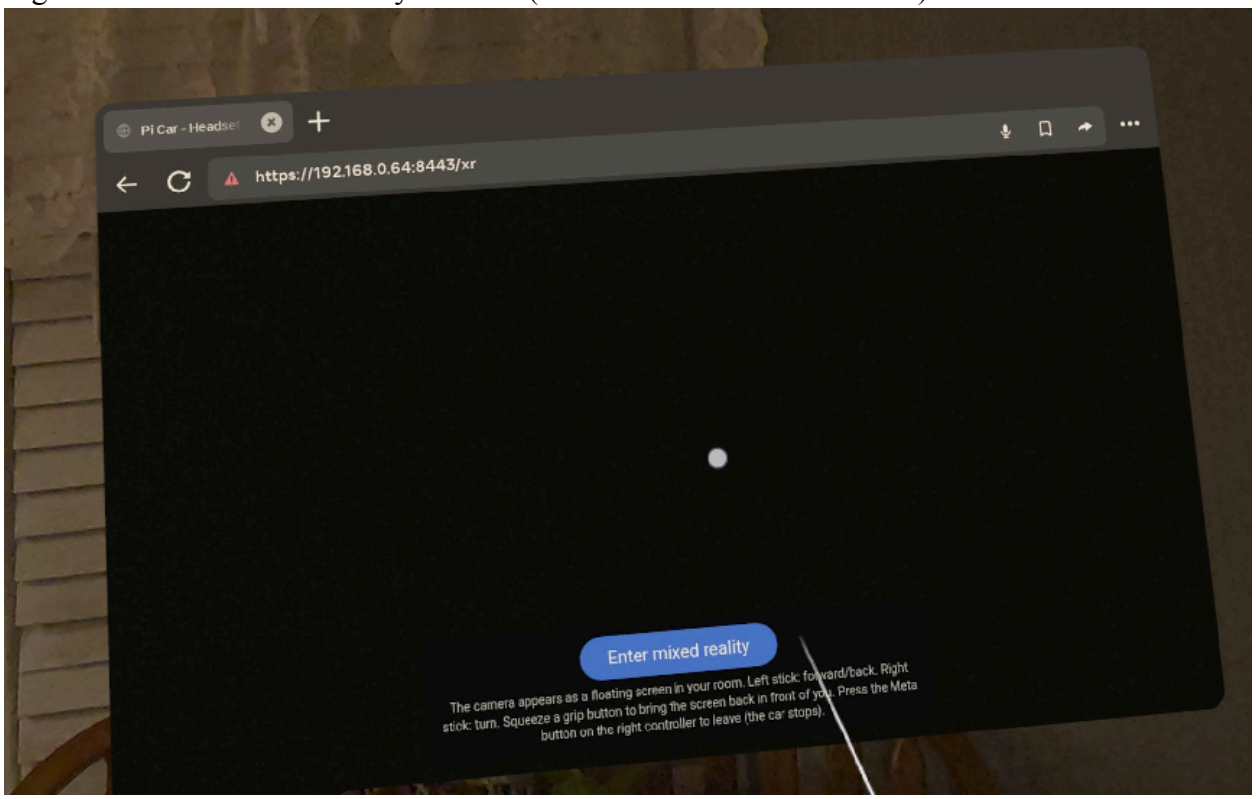
This page is where the user can access the “Browser” app that will allow them to enter the URL that will send them to the application hosted at that address.

Browser with URL to connect with the robot:



This is what the Browser application looks like when opened. The “No internet connection” is because our router does not have access to the internet to avoid interfering with other peoples’ internet traffic. Because all of our systems are on the same network, they don’t need WiFi to access each other, they have a direct connection through the internet router. From here, all we do is enter the URL to access the robot’s interface. Our URL is of the format “https://<IP address of the Raspberry Pi>:<Port Number>/xr”. In our case: “https://192.168.0.64:8443/xr”.

Page with “Enter Mixed Reality” Button (where the URL takes the user):



This is what the starting page of the robot interface looks like at the time of writing this document. Below the “Enter mixed reality” button are basic instructions on how to use the controllers to navigate the experience. Once here, all the user has to do is click the button to access the robot’s control page.

Robot Control Page:



* This will be a black box until the camera feed is sent over the network. (If possible, this will be re-submitted when that is completed)
From here, the user is able to control the robot with the joysticks and view the sensor data displayed to them. Outside of that the only other thing a user can do from here is exit the page by using the Meta Menu to navigate away from the application, closing the page, taking off the headset, going into the SSH terminal and manually terminating the running servers, or simply shutting everything off.

Errors and Edge Cases:

This section describes the different things that may go wrong during the use of our product, the measures we took to mitigate or prevent them (if possible), and what the user should do if they come across them

Hardware:
If the user finds that they're having hardware issues (things won't turn on or run, motors turn awkwardly or are louder/quieter than usual, camera feed is extremely laggy (some lag is normal), etc.). Essentially, if there's an issue and the Raspberry Pi's name and IP address are visible on the network and the Pi can send and receive messages, the issue is likely with the hardware. As you'll find by reading below, most hardware solutions are solved easiest by replacing the component that is faulty or damaged. Even with the necessary expertise, fixing the components often requires more effort than it's worth. This does not cover extreme scenarios in which the user integrates more advanced hardware, in which case we are not qualified to suggest solutions. As for the solutions within our scope:

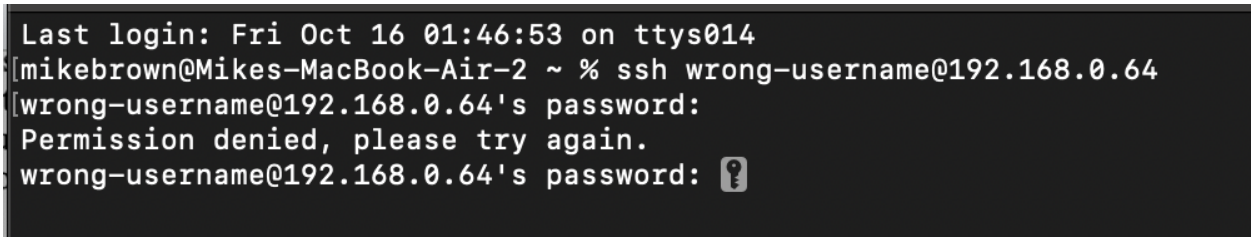
- **Dead/Faulty Batteries:**
  This can be diagnosed by observing that the robot is not turning on (no lights are visible on the computer boards or any of the on robot sensors). If the batteries are fully charged, the only way to isolate the issue is to use some sort of power meter to see if the battery's compartment is delivering power. The best solution is usually to just replace the batteries with newer ones to ensure that they aren't the issue. If the user is operating our instance of the project specifically, they will have access to multiple spare batteries located in the RUNE laboratory with the permission of Dr. Robin Hall. We recommend fully charging the batteries before use as motor drivers consume a significant portion of battery power even when idle, giving the user about 1 hour of operation time with fully charged batteries.

- **Dead Wires:**
  This can be diagnosed by observing that the robot is not turning on (no lights are visible on the computer boards or any of the on robot sensors) or if the robot is turning on but things like the motor or camera aren't receiving power. If the batteries are fully charged and functional and things still aren't receiving power this is the next likely issue assuming it isn't the component itself. One can use a power meter in a similar fashion to the solution above to verify the wire in question. There is no real technique for fixing this outside of replacing the wires themselves. Similar to the batteries, when using our platform, with the permission of Dr. Robin Hall or another Engineering faculty member, the user should be able to get access to wires and replace them.

- **Faulty Boards/Components:**
  - Similar to the others, this is best solved by replacing whatever component is causing the issue. If the system being controlled is more advanced than the one in this project, it is highly recommended that there be a means of testing subsets of the entire system such that if anything goes wrong, said tests can be run to isolate where the issue is coming from so that it can be addressed efficiently.

Digital/Networking Issues:
These issues usually come in the form of faulty network connections assuming it isn't just an error in the software being used in the case that the code from this project is altered. Assuming that is not the case, these issues can appear in a few ways, primarily with "choppy" controls and/or camera feed (repeatedly goes from working to not working, or simply just won't connect at all). To mitigate these issues, our team has implemented the use of a router to isolate the network and ensure that there is no traffic that would hinder our ability to connect the different parts of our system to each other. Any other issues would likely come from any software altered from what's found in this project. If it is, the best solution is to re-image the Raspberry Pi and re-upload the software.

Invalid Input:
The project is implemented such that the correct inputs are clear for the implementation we ourselves have made. Because this is a research project intended to be expanded upon and used in many different ways, the failure modes can be infinite depending on how other people choose to do so. For our project, there are virtually no spaces in which users can input anything that's not expected outside of maybe the terminal. In the terminal, they are expected to ssh into the Raspberry Pi using the correct username and password for the Pi. If anything other than those are used, they're connection will be rejected and they'll be presented with the following message until they enter the correct credentials:

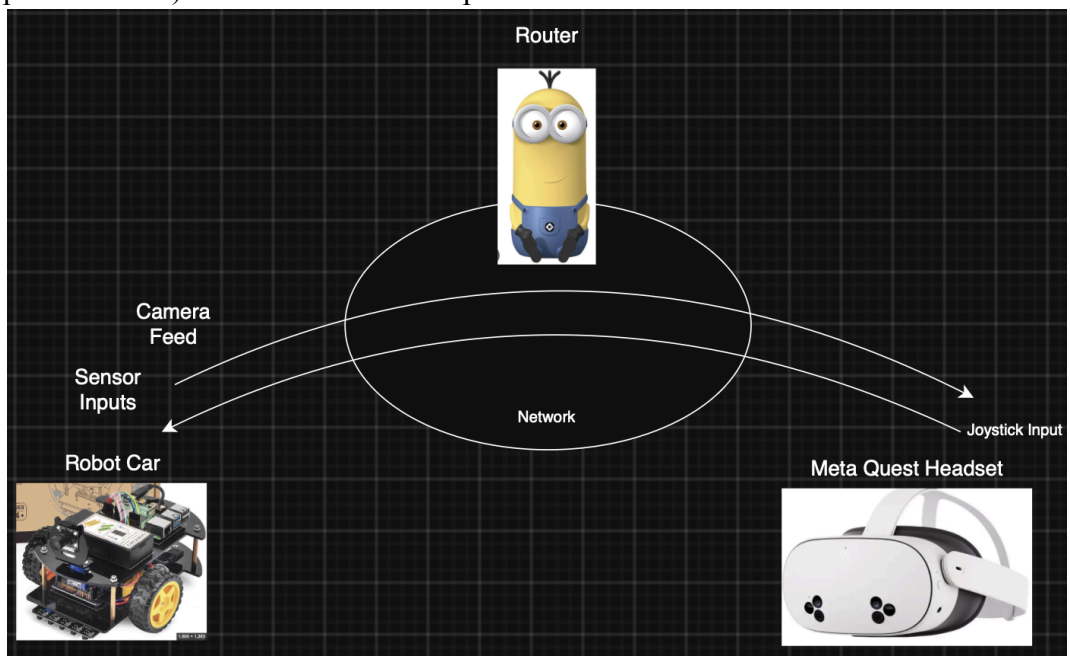


After this, the user is expected to run the servers using the commands displayed in the screen by screen walk through. Anything else is considered random input that will likely be handled by their computer's system code rather than ours. The same applies to the Meta Quest and all interfaces involved with it. In the browser, any URL that isn't the one mentioned will result in no connection, likely because it won't exist. Once on the start screen, any action that isn't closing the page or pressing the trigger on the "Enter mixed reality" button will be ignored. Once the user is in the control interface, nothing is recognized outside of actions that close the page and the throttle inputs. There is very little room for error or randomness that leads to alternative functions.

Technical Components:

Network Diagram:

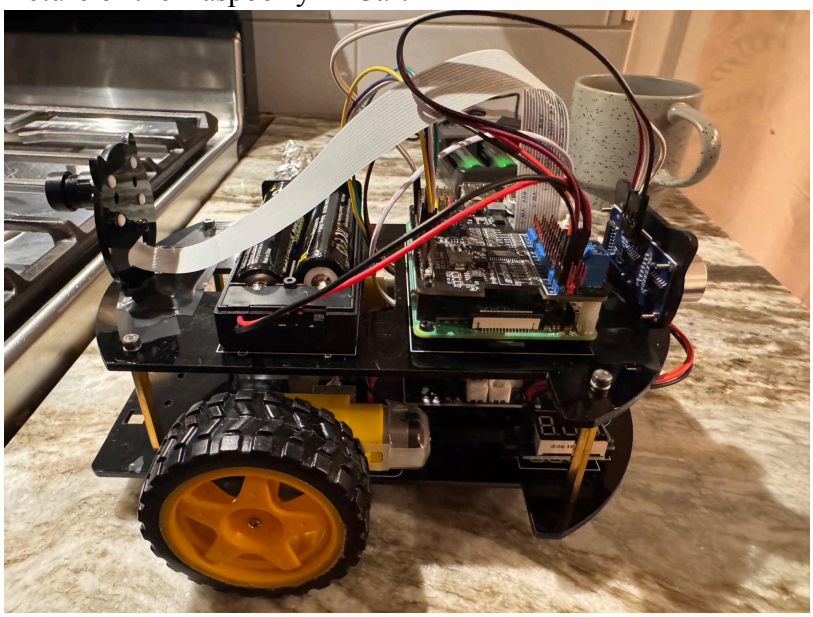
Fun Fact: In the electrical engineering community there is a router built like one of the minions from the Despicable Me series (Kevin). It's ironically one of the best routers on the market and our group wishes we found it before we started the project (we may end up getting it for final presentations). Our actual router is tp-link's AX1800 Dual Band Wi-Fi 6 Router



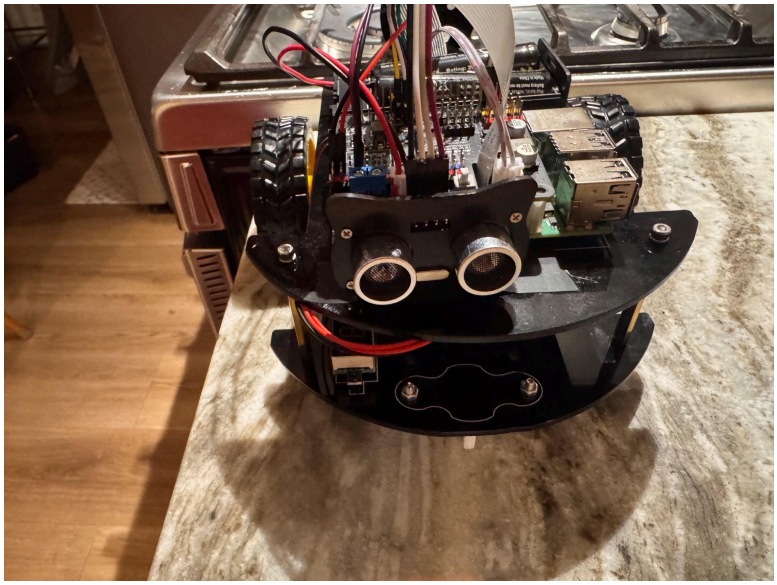
Picture of the Meta Quest 3 and its joysticks:



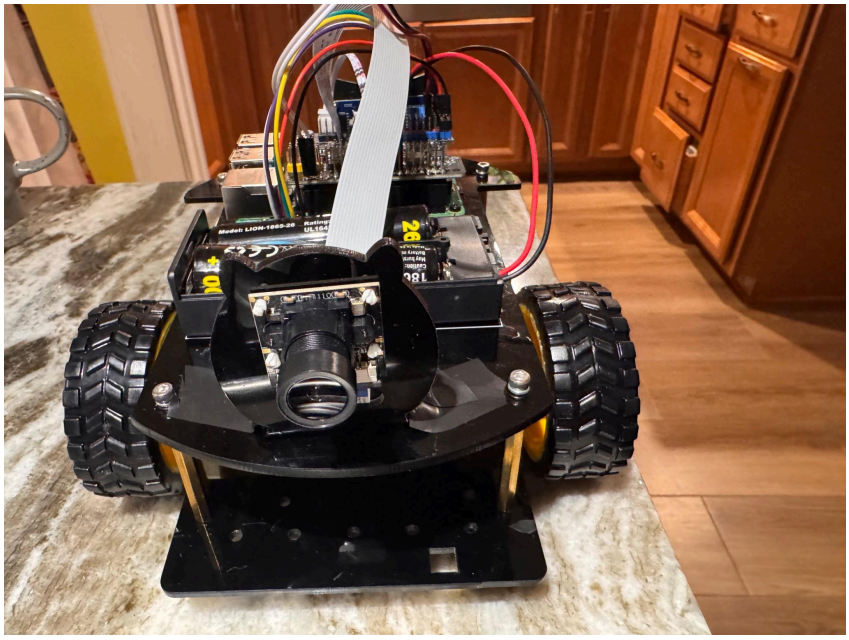
Picture of the Raspberry Pi Car:



Picture of the Ultra Sonic Sensor:



Picture of the Camera:



Picture of the Router:



Picture of the Ethernet Cable:



Picture of the Lipo Battery with Charger:



- **How does the joystick input get sent?**
  The /xr page running in the Quest browser has a loop that fires about 60 times a second. On each pass it asks the browser a simple question: where are the thumbsticks right now? The browser answers with four numbers, two for each stick, each between -1 and +1.

Centered is 0. Pushed fully forward is -1 (the Quest reports forward as negative, which is why the server flips the sign later). Fully right is +1. The page packages those four numbers into a tiny text message, something like {"lx":0, "ly":-0.8, "rx":0.3, "ry":0}, and pushes it through a WebSocket to the Pi on port 8765. A WebSocket is just a pipe that stays open, unlike a normal web request, which opens, delivers one thing, and closes. Keeping it open is what makes this feel instant instead of laggy. That's roughly 60 messages per second, each a few dozen bytes, small amounts of data. The video is thousands of times heavier. The page sends on every pass whether or not anything changed, which matters for the next part.

- **How does the joystick input get translated into motor inputs to move the actual car?**

drive_server.py on the Pi is listening on the other end of that pipe. For each message it does four things.

Arcade mixing. Two sticks, two motors, but not one stick per motor. The left stick's vertical axis is throttle, the right stick's horizontal axis is turn. The server combines them: left motor = throttle + turn right motor = throttle - turn. Push forward with no turn and both get the same number, so the car goes straight. Add some right turn and the left wheel gets more, the right gets less, and the car arcs right. Turn with no throttle and the two come out opposite, the car spins in place. It's called arcade mixing because it's how an arcade cabinet maps a single stick to a tank drive.

Sign and scale. The result is a number from -1 to +1. The sign becomes direction; the size becomes speed. If LEFT_REVERSED is set, that wheel's sign gets flipped, the fix for a motor wired backwards, done in software rather than by re-soldering. The magnitude is then squeezed between MIN_SPEED (0.25, below which the motor just hums and doesn't turn , not enough torque to overcome its own friction) and MAX_SPEED.

Out to the hardware. Direction and speed travel on separate paths, which is the part that usually surprises people. Direction goes over four GPIO pins, 23, 24, 27, 22, to the L298N motor driver. Each motor has two of them, and which one is high decides which way the motor spins. It's a switch, nothing more. Speed goes over I2C to the PCA9685 chip on the HAT. The Pi tells it something like "channel 0, 70% duty cycle" and the chip starts flipping that line on and off a thousand times a second, 70% of the time. The motor can't respond that fast, so it averages it out and behaves like it's getting 70% power. That's PWM. The important bit: once told, the PCA9685 keeps doing it on its own. The Pi doesn't babysit it. Python can be busy, the network can hiccup, and the motors keep running at the last commanded speed. That independence is what makes the next piece necessary.

The watchdog. The server tracks when the last message arrived. If 0.3 seconds pass with nothing, headset crashed, WiFi dropped, you walked out of range, it zeroes both motors. Without it, that last "full speed forward" would stay latched in the PCA9685 and the car

would drive itself into a wall. This is also why the page sends continuously rather than only on change: the steady stream is the “heartbeat” the watchdog is listening for.

- **How does the camera feed get back to the Meta Quest?**
  Four stages. First, the CSI ribbon cable carries raw sensor data straight into the Pi’s camera interface, not USB, but a dedicated path. The Pi’s image processor turns that raw sensor data into a normal image: white balance, exposure, color. Then each frame gets compressed to a JPEG.

  camera_server.py serves those JPEGs over HTTP using MJPEG, Motion JPEG. It’s exactly what it sounds like: a stream of complete JPEGs, one after another, forever. The browser displays each as it arrives and you perceive motion.

  MJPEG is inefficient compared to real video codecs, because every frame is complete and independent, no “only what changed since last frame” shortcut. But it’s dead simple, every browser handles it natively, and it has almost no buffering latency. For teleoperation, latency beats file size. A compressed stream that arrives 300ms late is useless when you’re moving.

  The /xr page puts that stream on a flat panel floating in the Quest’s passthrough view. You see your real room, with a video panel hanging in it. The panel isn’t stereoscopic, there's one camera, one image, both eyes see the same thing. You lose depth perception, which is worth knowing before you try to judge distances while driving.

  It’s important to note that the two parts are independent of each other. The video and controls live on separate ports that don’t communicate whatsoever, which also means that one can fail without the other. Our application can access both because they’re both held by the browser.

Glossary:

Microcontroller:
A microcontroller is a very small computer on an integrated circuit that can decide what processes happen and when on a variety of electronic devices, such as microwaves, thermostats and washing machines. Since it is very small, it’s not as powerful as most of the bigger computers are as of this point in time due to having a small amount of memory and limited software capabilities, but on the brighter side of things, it doesn’t use as much electricity as a bigger computer would, either.

Raspberry Pi:
A more powerful genre of microcontrollers(depending on what version you buy) that can do almost everything a regular computer can do while also not costing as much money to buy as that said computer. In this project, the Raspberry Pi will function as a translator between the VR headset and the motors.

Single Board Computer:
Essentially a miniature computer, similar to the ones people use almost every day, except that it is much less powerful (e.g. smaller amounts of memory, more limited software capabilities, etc) with the added benefit of not requiring as much electricity to use.

VR:
An acronym for Virtual Reality, which refers to the concept of an electronic device(typically a headset) using components such as cameras and sensors to run a program that overrides the real world camera feed and replaces it with a fully digitalized scene. In this project, this term will be used to refer to virtual reality headsets such as the Meta Quest and Apple Vision Pro.

AR:
Similar to Virtual Reality in concept, but this time, Augmented Reality(AR) only focuses on making digital objects appear as though they are actually in the real world via the aforementioned cameras and sensors.

Apple Vision Pro:
Apple's proprietary virtual reality platform(which may also double as a Mixed Reality (Virtual Reality(VR) + Augmented Reality(AR)) platform when used for activities outside the scope of this project. Compared to most other virtual reality headsets, all you need to do to control what the Vision Pro does is move your eyes and/or hands, or speak into the headset's microphone with your voice.

Meta Quest:
Meta's proprietary virtual reality platform that no longer requires a PC to run as of the Meta Quest 2's release. It was called the Oculus Quest about a decade ago, but had its name changed when Facebook's parent company changed its name to Meta in 2022 after buying the Oculus VR company on March 25, 2014. Compared to most other virtual reality headsets, you can download the Meta Horizon app on your smartphone to set it up instead of having to rely on a PC.

Router:
Manages data traffic between devices on a network. Will be used in this project to create an isolated network to minimize connection issues between the headset and the device.

IMU (Inertial Measurement Unit):
A small sensor chip that measures how a device is moving and tilting. It usually combines an accelerometer (measures changes in speed) and a gyroscope (measures rotation). Some also include a magnetometer, which works like a compass. In this project, it would report the robot's orientation.

WebXR Gamepad API:
A set of tools built into web browsers that lets a webpage read VR controller inputs, such as joystick position, button presses, and hand pose. This is how the app running in the Quest browser reads the controllers.

Hand Pose:
The position and orientation of the user's hand or controller in 3D space, as tracked by the headset.

JavaScript:
The programming language that runs inside web browsers. Here, it runs in the Quest browser to read the controllers and send commands.

Python Server:
A program written in Python that runs on the Raspberry Pi. It listens for incoming commands from the headset and translates them into motor instructions.

WebSocket:
A type of network connection that stays open, so data can flow in both directions continuously. It is well suited to real-time control because each joystick update doesn't need a brand-new connection.

WiFi (2.4GHz / 5GHz):
Wireless networking. The 5GHz band is faster and less crowded than 2.4GHz, which helps with low-delay video, but it has a shorter range and is worse at passing through walls.

Network Stack:
The layers of software in a device that handle sending and receiving data over a network.

Network Congestion:
Slowdowns that happen when too many devices or too much data are competing on the same network.

Isolated Network:
A private network, created here by the dedicated router, with only the project's devices on it. It avoids interference from other traffic.

Latency:
The delay between an action and its result, such as the time between moving the joystick and the robot responding, or between the camera capturing a frame and it appearing in the headset. Low latency is critical for comfortable VR control.

CSI Camera:
A camera that plugs directly into the Raspberry Pi's dedicated camera port through the Camera Serial Interface (CSI). It is faster and more efficient than a USB webcam because it connects straight to the Pi's image-processing hardware.

Ribbon Cable:
A flat, wide cable made of many thin wires side by side. It connects the CSI camera to the Raspberry Pi.

PWM (Pulse Width Modulation):
A way of controlling power by switching a signal on and off very quickly. The share of time the

signal stays "on" (the pulse width) sets how much power gets through. A wider pulse makes a motor spin faster, and a narrower pulse makes it spin slower.

PWM HAT (PCA9685):
An add-on board that sits on top of the Raspberry Pi. "HAT" stands for Hardware Attached on Top. The PCA9685 chip generates steady, precise PWM signals on many channels at once, so the Pi doesn't have to produce the timing itself.

H-Bridge:
A circuit that can reverse the direction of the current flowing through a motor, which lets the motor spin both forward and backward. Motor driver boards are built around one or more H-bridges.

Motor Driver Board (Model X):
A board that takes the low-power control signals from the PWM HAT and switches the much larger battery current the motors need. The Pi can't power motors directly, so the driver board acts as the middleman.

Gear Motor:
An electric motor with a built-in gearbox. The gears trade speed for torque, which gives the motor enough turning force to move the robot's wheels.

I2C Bus:
A simple two-wire communication system that lets the Raspberry Pi talk to multiple chips, such as the PWM HAT or an IMU. Each device on the bus has its own address, so they can share the same wires.

ISP (Image Signal Processor):
A part of the Pi's chip that turns the raw data from the camera sensor into a usable image. It handles tasks like adjusting color, brightness, and noise.

Ethernet Cable:
A wired network cable. It is useful for setting up the Pi or router when a wireless connection isn't working yet.

H.264 Encoder:
Hardware or software that compresses video using the H.264 standard. It makes the video small enough to send over WiFi quickly. The Pi 4 has a dedicated chip that does this.

Decode:
Decompressing the encoded video on the receiving end so it can be displayed. Here, the Quest browser does the decoding.

Camera Feed:
The live video stream from the robot's camera.

Throttle:
The control that sets forward and backward speed. Here, it is the left joystick.

Yaw:
Rotation left or right around the vertical axis, meaning turning in place or steering. Here, it is controlled by the right joystick.

Orientation:
Which way the robot is facing and how it is tilted. It is usually described as yaw (turning), pitch (nose up or down), and roll (tilting side to side).

Project Management

GitHub Repository:
An online folder where the project's code is stored, versioned, and shared among team members.

Sprint:
A fixed block of time, often one to two weeks, focused on completing a specific set of goals for a project.

Arcade Mixing
A way of converting two joystick inputs into two motor speeds. One axis (usually a stick's vertical) sets throttle, another (usually a stick's horizontal) sets turn. The two are combined for each side: left = throttle + turn, right = throttle - turn. Equal values send the vehicle straight; unequal values make it arc; opposite values spin it in place. Named for arcade driving games, which mapped a single stick to a two-motor drive this way.

Arcade Cabinet
A standalone coin-operated video game machine, a full-size enclosure housing a screen, controls, and the game hardware. Common in arcades from the late 1970s onward. Its physical controls (a joystick and buttons) are what gave "arcade-style" control schemes their name.

Soldering
Joining two pieces of metal by melting a filler metal (solder) between them, which cools into a permanent electrical and mechanical connection. Standard for attaching components to circuit boards and wires to terminals. Undoing a solder joint requires reheating it, which is why fixing a reversed motor in software is usually preferable to rewiring it.

USB
Universal Serial Bus, a general-purpose standard for connecting peripherals to a computer, carrying both data and power over one cable. Deliberately generic, so it works with almost any device, but that generality costs bandwidth and adds latency compared to a dedicated connection. The Raspberry Pi's CSI camera port bypasses USB entirely and wires the sensor closer to the processor.

JPEG
A compressed image format, and the compression method behind it. It discards detail the human eye is least likely to notice, fine color variation especially, to make files much smaller. "**Lossy**": the discarded information is gone permanently. Quality is adjustable, trading file size against visible artifacts.

MJPEG
Motion JPEG, video sent as a continuous sequence of complete, independent JPEG images. Each frame stands alone, with no reference to the frames around it. This makes MJPEG larger than modern video formats for the same quality, but simple to produce, universally supported by browsers, and very low latency, since no frame has to wait for another to be decoded.

Codec
Coder-decoder: the paired algorithms that compress a video or audio stream for transmission and reconstruct it on the other end. Modern video codecs (H.264, H.265, AV1) achieve high compression by storing only what changed between frames, which requires buffering several frames at once, efficient for streaming, but it adds delay that works against real-time control.

Stereoscopic
Presenting a slightly different image to each eye, mimicking the offset between human eyes, so the brain perceives depth. Requires two cameras separated by roughly the distance between a person's pupils. A single camera shown to both eyes is monoscopic: the image appears flat, and judging distance depends on indirect cues like relative size and motion.

Switch
A component with two states , closed (current flows) or open (it doesn't). No intermediate positions. In a motor driver, switches set direction by determining which way current passes through the motor. Speed cannot be set this way, since a switch has no "partly on," which is why PWM, rapidly toggling a switch and varying how long it stays on, is used instead.

SSH
A way to type commands into one computer from another. You open a terminal on your Mac, connect to the Pi over the network, and from then on everything you type runs on the Pi, its files, its programs, its hardware. The Pi needs no keyboard, mouse, or monitor of its own; it just needs to be powered on and on the network. “Secure” means the connection is encrypted, so nobody else on the network can read what you’re sending. The command ssh capstone@192.168.0.64 means “log in as the user *capstone* on the machine at that address.”

Terminal
A window where you type commands as text instead of clicking buttons. You type a command, press Enter, and the computer prints its response below. Every tool on the Pi can be driven this way, and many have no clickable interface at all. Each terminal window is a separate session,

this is why the project runs camera_server.py in one window and drive_server.py in another. A program keeps control of its window while it runs, printing status as it goes, so two simultaneous programs need two windows.

Browser
The program you use to view web pages, Chrome, Safari, Firefox, and the one built into the Quest. It fetches a page from a server, interprets the HTML and JavaScript inside, and displays the result. The important part for this project: a browser doesn't just display pages, it *runs* them. The /xr page's code executes inside the Quest's browser, which is what lets it read the controllers and open a connection back to the Pi. No app installation involved, the browser is the app.

KB (Short for kilobyte)
A unit of data size. One kilobyte is about a thousand bytes; a byte is roughly one character of text. In the log line 19 KB per frame, it means each JPEG image the camera produces is about 19,000 bytes. Multiply by the frame rate to get the network load: 19 KB × 16 frames per second ≈ 304 KB per second, or about 2.4 megabits per second over WiFi. Sharper or larger images produce bigger frames, which is why raising the resolution raises the bandwidth.

Certificate (in the context of websites and servers)
A file that proves a server is who it claims to be, and supplies the keys that encrypt the connection. Normally a trusted company issues it and your browser accepts it silently. The Pi's certificate is *self-signed*, generated by the Pi, vouching for itself, so no browser trusts it automatically, and you get a warning you have to click past. The encryption still works; what's missing is third-party verification of identity, which on your own lab network is not a meaningful loss. The reason the project needs one at all is that WebXR refuses to start on an unencrypted page, so HTTPS is mandatory even on a local network.

Special Acknowledgements:

Thank you to Dr. Prathuri, Dr. Hall, and Dr. Benedict for allowing us to participate in this project and allowing us to use their lab space and equipment for all of the project's needs.